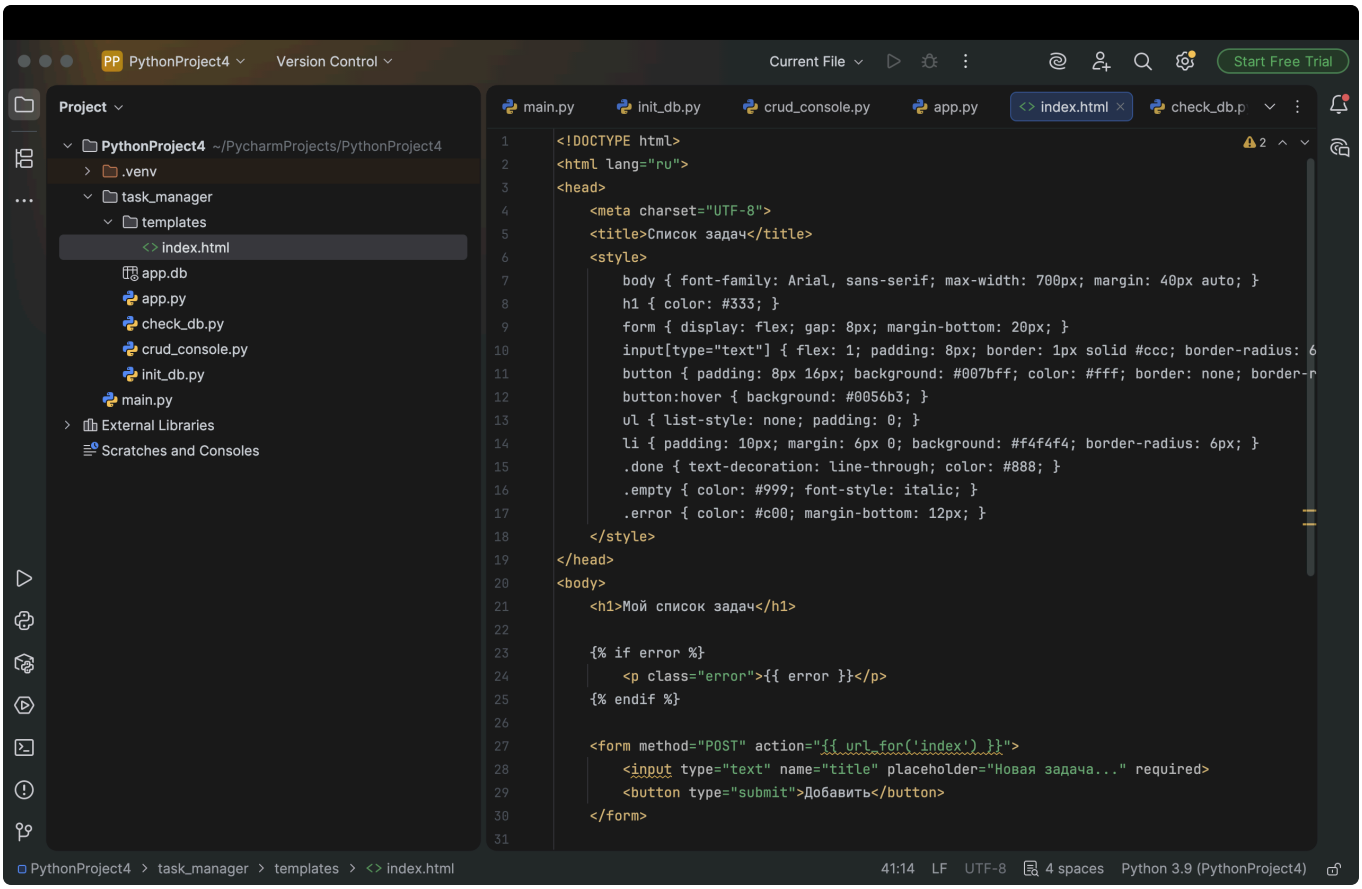
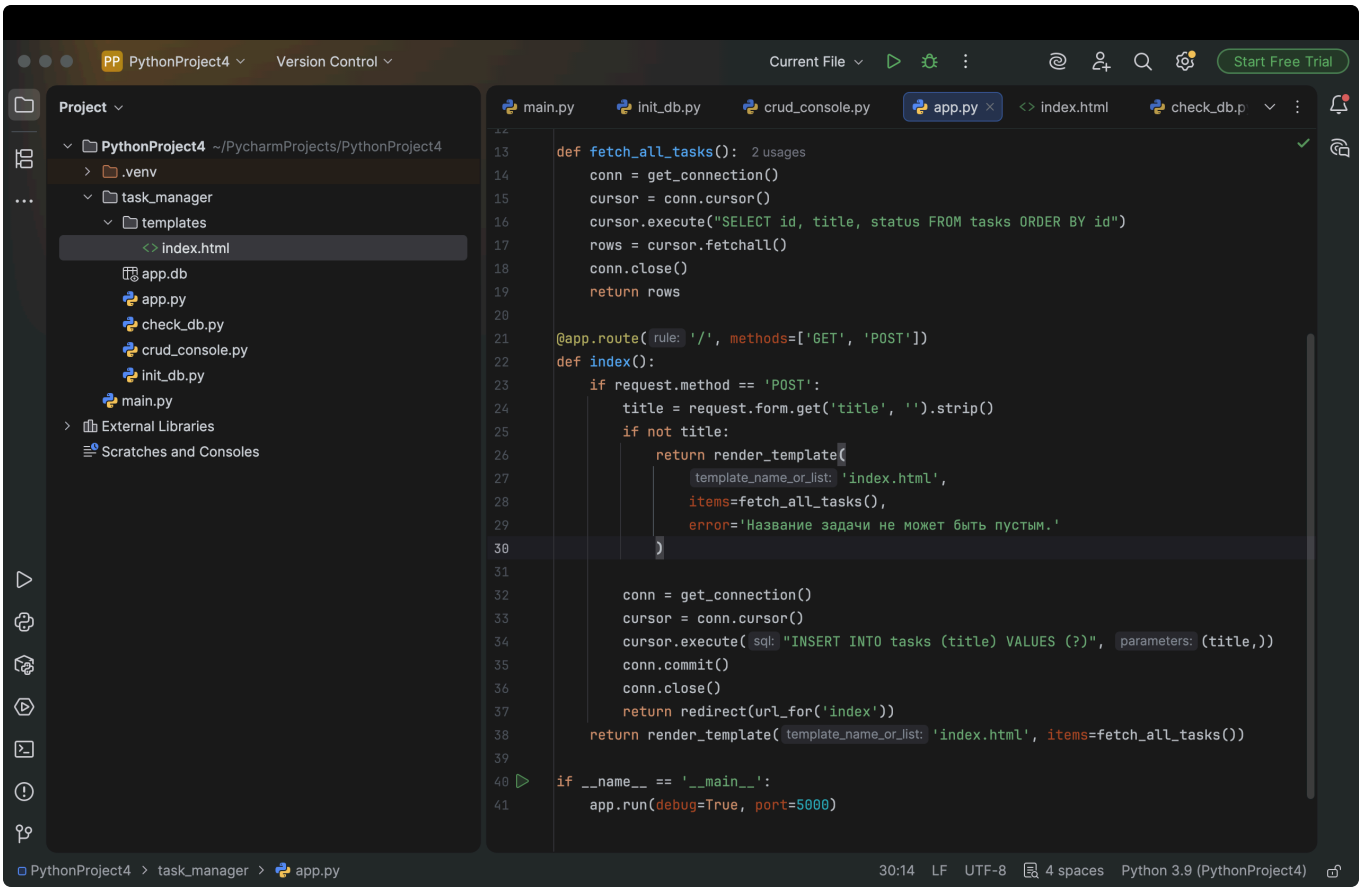
Интеграция Flask и SQLite (Добавление данных)

Ястребкова Лилия ЗИСП11-44

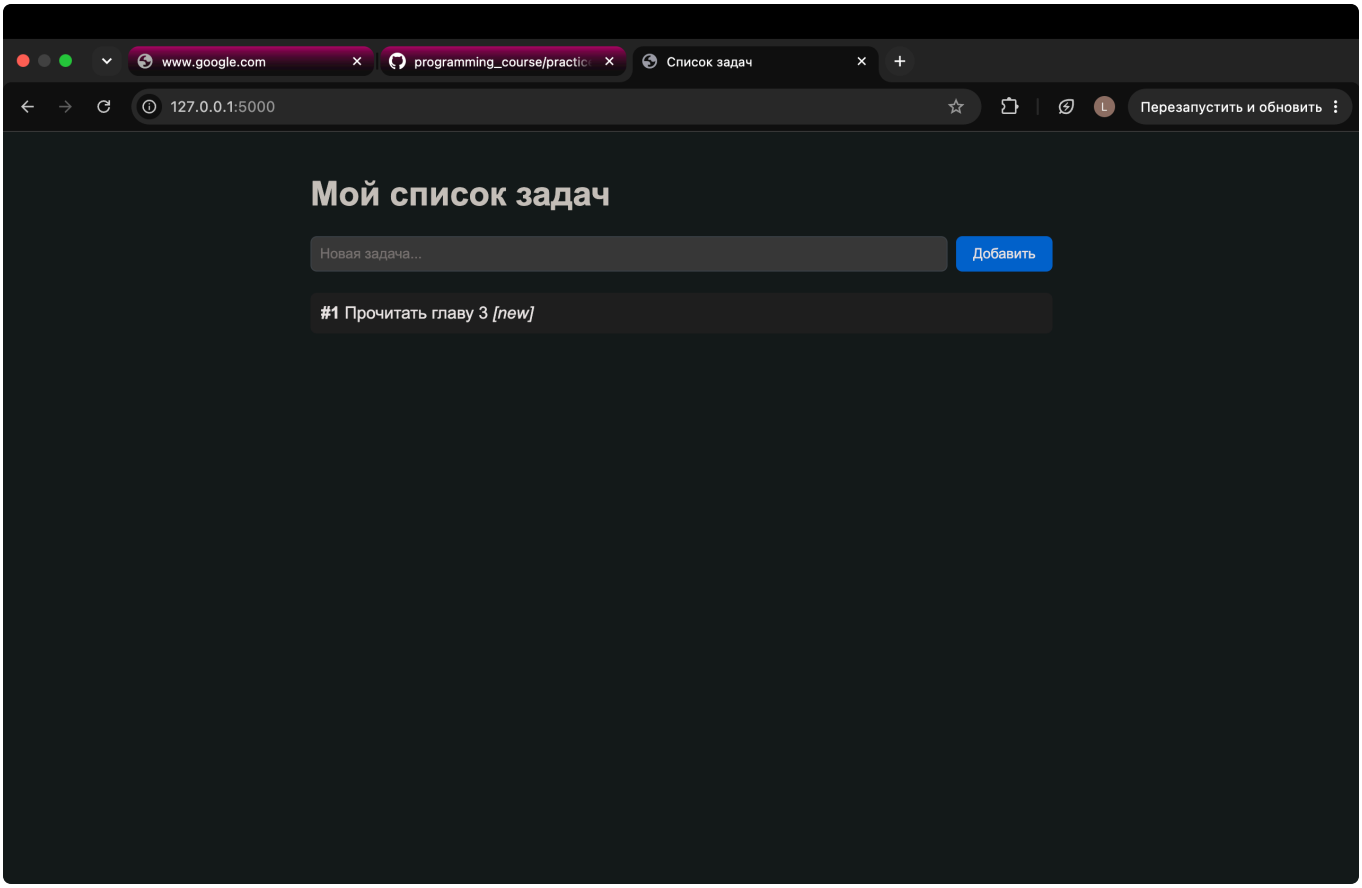
Шаг 1. Обновляем шаблон `templates/index.html` — добавляем форму

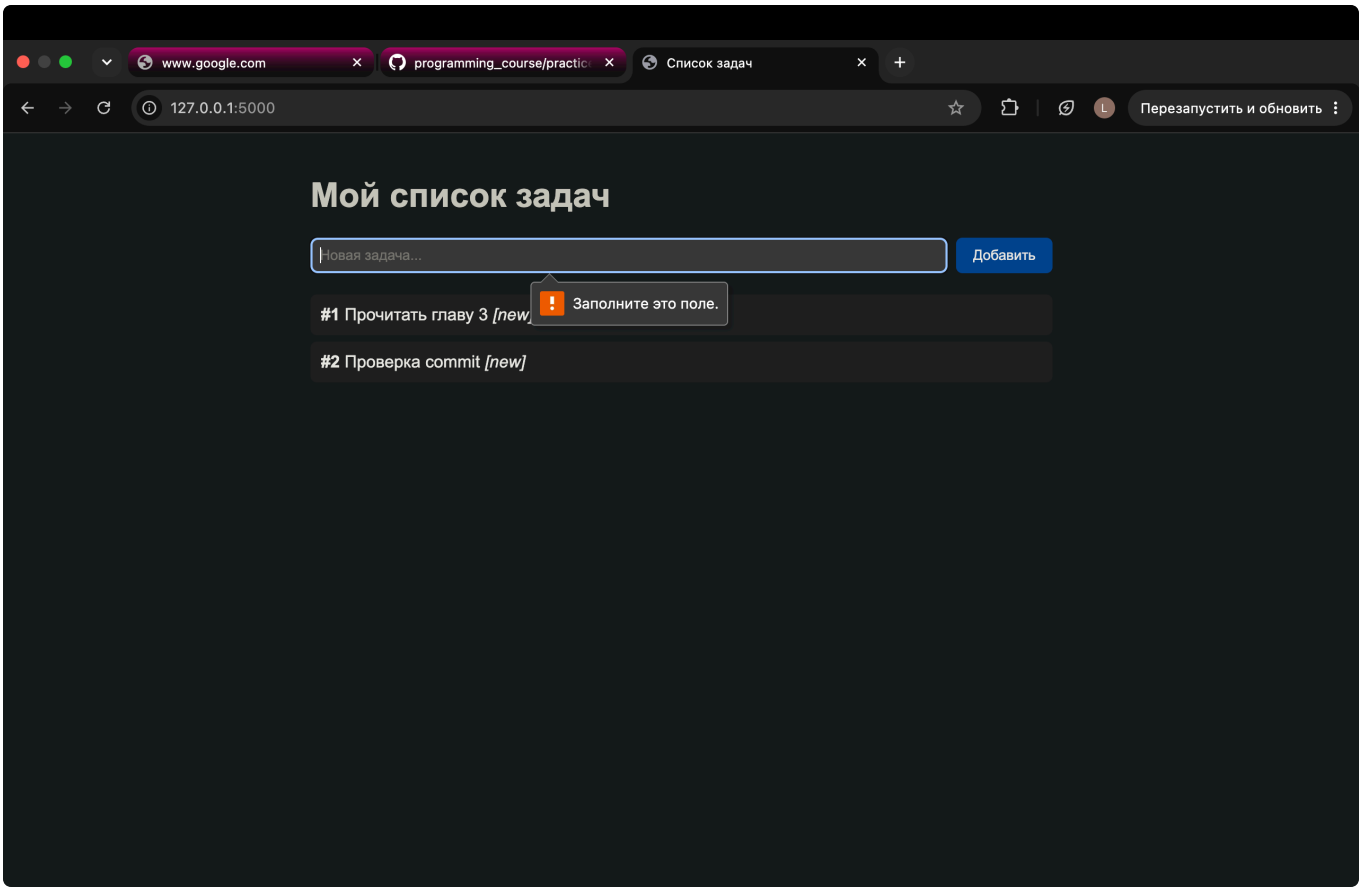


Шаг 2. Обновляем `app.py`: обработка POST и INSERT



Шаг 3. Запуск и проверка





1. Чем POST принципиально отличается от GET и почему формы отправляют именно POST?

`GET` предназначен для получения данных: параметры передаются в URL, видны в адресной строке, кэшируются браузером, а повторный запрос безопасен (идемпотентен). `POST` предназначен для отправки данных на сервер: они идут в теле запроса и не видны в URL. Главное различие в семантике: GET не меняет данные на сервере, POST - меняет (создаёт или обновляет записи). Формы отправляют POST именно потому, что добавление задачи - это изменение данных: запись создаётся в базе, и повторный запрос даст другой результат. Для операций, меняющих состояние, POST и предназначен.

2. Что произойдёт, если убрать `methods=['GET', 'POST']` из маршрута?

По умолчанию Flask-маршрут принимает только GET. При отправке формы методом POST браузер получит ответ **405 Method Not Allowed** - метод не разрешён для этого URL. Страница будет открываться, но форма работать перестанет. Поэтому в маршруте обязательно нужно указать `methods=['GET', 'POST']`.

3. Зачем нужен паттерн Post/Redirect/Get? Что сломается без него?

Без него повторная отправка формы приводит к дублированию данных. Если после INSERT просто вернуть HTML-страницу, то при нажатии F5 браузер повторит последний запрос - это POST, а значит снова выполнится INSERT и появится копия задачи (каждое обновление страницы - новая запись). Паттерн PRG решает это: после успешного POST сервер отвечает редиректом (HTTP 302) на GET-страницу. Браузер переходит по адресу редиректа, получает свежий список, а повторное обновление теперь повторяет безопасный GET - дублирования нет.

4. Почему `url_for('index')` лучше, чем жёсткая строка `'/'`?

`url_for` строит URL по имени функции-обработчика, а не по тексту маршрута. Если маршрут изменится (например, с `/` на `/tasks`), `url_for('index')` подставит новый адрес автоматически, а жёстко прописанная строка `'/'` останется старой и приведёт к неработающей ссылке. Это делает код устойчивым к изменениям и избавляет от правок во многих местах - URL строится в одном месте и используется везде.

5. Почему `request.form.get('title', '')` безопаснее, чем `request.form['title']`?

`request.form` работает как словарь. Обращение по ключу `request.form['title']` выбрасывает исключение `BadRequestKeyError`, если поля в форме нет. Метод `.get('title', '')` в этом случае просто вернёт значение по умолчанию `''` — программа не упадёт, а сможет обработать ситуацию (например, показать ошибку валидации). Особенно это важно при обработке форм с возможными пропущенными полями или при отправке запросов в обход браузера.

6. Зачем нужна серверная валидация, если в HTML уже есть `required`?

`required` - это только удобство для пользователя и проверка на стороне браузера, а не защита. Злоумышленник может отправить POST-запрос напрямую через curl, Postman или скрипт, вообще не загружая форму, - тогда атрибут `required` не сработает. Единственная надёжная проверка - на сервере: в Python-коде проверять данные, пришедшие из формы (`if not title:`), и отклонять некорректные. Браузерная валидация экономит запросы и улучшает UX, серверная - гарантирует корректность данных в базе.

7. В каком порядке идут транзакции при добавлении задачи через форму?

Порядок такой:

1. Пользователь заполняет форму и нажимает «Добавить» - браузер отправляет `POST /`.
2. Flask вызывает обработчик; `request.form.get('title')` читает значение из формы.

3. Серверная валидация: если поле пустое — возврат формы с ошибкой, без записи в базу.
4. `conn = sqlite3.connect(...)` - открытие соединения.
5. `cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))` — параметризованная вставка (изменение «в черновике»).
6. `conn.commit()` - фиксация на диск.
7. `conn.close()` - освобождение файла базы.
8. `redirect(url_for('index'))` - редирект на GET `/`.
9. Браузер делает `GET /`, страница рендерится с обновлённым списком.